

Unidad 3. Excepciones, bucles, arrays y métodos

Control estructurado de errores con excepciones (try-catch-finally), estructuras iterativas (while, do-while, for, for-each), estructuras de datos homogéneas (arrays y matrices), procesamiento avanzado de texto con StringBuilder, y diseño modular de software con métodos, paso de parámetros y recursividad en Java 25 LTS.



MÓDULO PROFESIONAL

Programación (0485) — 1º DAM

DURACIÓN ESTIMADA

28 horas lectivas

RESULTADOS DE APRENDIZAJE

RA1, RA2, RA3, RA6 (Estructuras de control, arrays, métodos y excepciones)

ENTORNO OFICIAL

IntelliJ IDEA / Java 25 LTS

Hasta este punto de tu formación como desarrollador de software, tus programas han seguido una lógica eminentemente lineal y selectiva: el código se ejecutaba de arriba abajo y tomaba bifurcaciones mediante sentencias `if-else` o `switch`. Sin embargo, la auténtica potencia de la computación reside en su capacidad para **procesar millones de datos en fracciones de segundo mediante la repetición controlada, blindarse ante fallos imprevistos y estructurar algoritmos complejos en piezas reutilizables y modulares**.

En esta unidad nos adentramos en el núcleo de la programación estructurada con **Java 25 LTS** (con compatibilidad plena para Java 21 LTS) y nuestro entorno de trabajo oficial, **IntelliJ IDEA (Visual Studio Code)**:

1. **Gestión de Excepciones:** Cómo evitar que nuestras aplicaciones se cuelguen ante entradas incorrectas del usuario, recursos no disponibles o errores aritméticos mediante bloques `try-catch-finally` y `try-with-resources`.
2. **Sentencias de Repetición (Bucles):** Automatización iterativa con `while`, `do-while` y `for`, junto con técnicas de control de flujo (`break`, `continue` y variables centinela).
3. **Arrays y Matrices:** Almacenamiento contiguo y masivo de información en memoria, recorridos optimizados, algoritmos clásicos de búsqueda y ordenación, y herramientas de la clase estándar `java.util.Arrays`.
4. **Procesamiento de Cadenas:** Técnicas de recorrido e inspección sobre textos y optimización de memoria mediante la clase `StringBuilder`.
5. **Programación Modular (Métodos):** Descomposición de problemas según el paradigma de "divide y vencerás", paso de parámetros por valor, mutabilidad en memoria Stack y Heap, sobrecarga y algoritmos recursivos.
6. **Expresiones Regulares (Regex):** Validación profesional de patrones sintácticos complejos (DNI, correos, formatos numéricos).

1. Introducción a la Programación Estructurada y Control Avanzado

En los orígenes de la informática moderna, el control de flujo en lenguajes primitivos dependía en gran medida de saltos incondicionales (el tristemente célebre `GOTO`), lo que provocaba el llamado "código espagueti": un laberinto indescifrable de saltos cruzados imposible de depurar y mantener.

En 1966, los matemáticos Corrado Böhm y Giuseppe Jacopini demostraron el **Teorema del Programa Estructurado**: cualquier algoritmo computable puede expresarse combinando únicamente **tres estructuras fundamentales**:

Pilares de la Programación Estructurada

- 1. Secuencia → Instrucción tras instrucción de forma consecutiva
- 2. Selección → Bifurcaciones lógicas (`if-else`, `switch`) [Visto en la Unidad 2]
- 3. Iteración → Repetición controlada de bloques (`while`, `do-while`, `for`) [Núcleo de esta Unidad]

A estos tres pilares clásicos, la ingeniería de software moderna ha añadido dos disciplinas de vital importancia:

- **La Modularidad (Métodos):** Aislar responsabilidades en subrutinas independientes, testables y autoexplicativas.
- **La Robustez y Manejo de Errores (Excepciones):** Mecanismos formales para separar la lógica de negocio habitual de los caminos de contingencia ante fallos en tiempo de ejecución.

2. Gestión Profesional de Excepciones y Robustez de Software

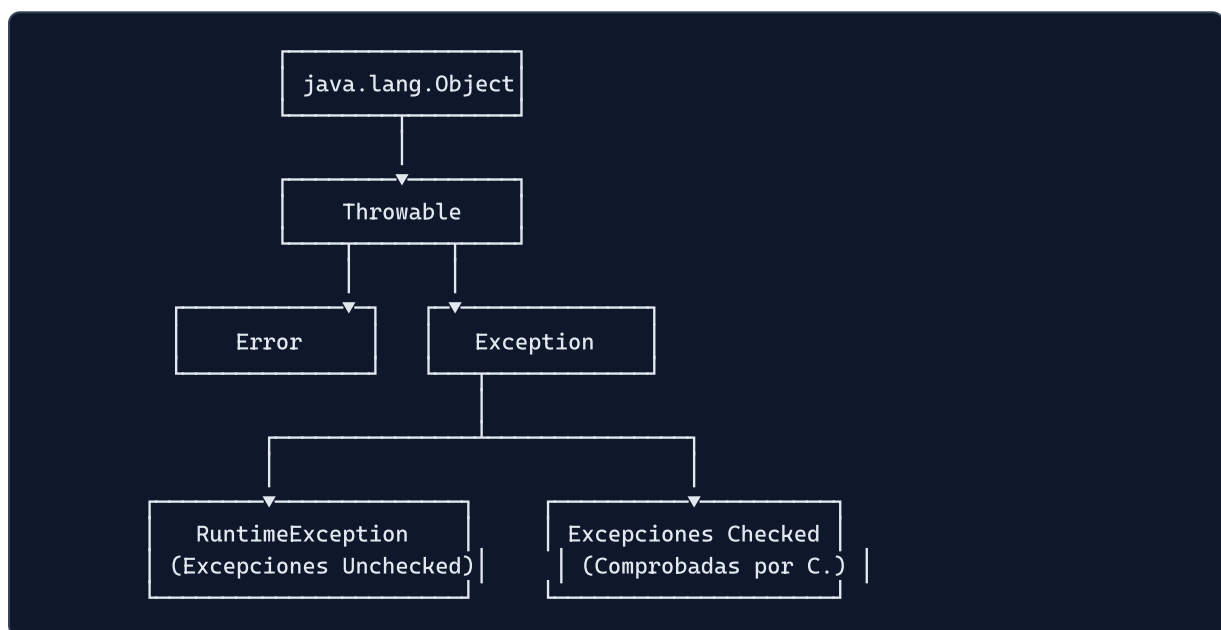
Todo programa informático opera en un entorno imperfecto: los usuarios introducen letras cuando se les piden números, los discos duros se llenan, las conexiones de red se interrumpen y los divisores pueden ser cero.

Un programa amateur se “estrella” arrojando un críptico error rojo en la consola. Un software profesional **anticipa las contingencias, intercepta los errores y reacciona de forma controlada**.

2.1 Qué es una Excepción y la Jerarquía `Throwable` de Java

En Java, un error en tiempo de ejecución no es una simple señal numérica del sistema operativo: **es un objeto de pleno derecho** que se instancia en la memoria Heap y se “dispara” (*throw*) hacia arriba en la pila de llamadas de la JVM.

Todos los objetos de error derivan de la superclase `java.lang.Throwable`, dividida en dos ramas cruciales:



1. **Error**: Fallos catastróficos a nivel de hardware o de la propia JVM (como `OutOfMemoryError` o `StackOverflowError`). El programador **no debe intentar capturarlos**, ya que la máquina se encuentra en un estado irrecoverable.
2. **Exception**: Situaciones anómalas de las que un programa bien diseñado sí puede recuperarse. Se subdividen en dos grandes familias:
 - **Excepciones no comprobadas (Unchecked / RuntimeException)**: Suelen derivar de errores de programación lógica (como `NullPointerException`, `ArithmeticException`, `ArrayIndexOutOfBoundsException`, `InputMismatchException`). El compilador no te obliga a capturarlas explícitamente, pero debes prevenirlas con validaciones o capturarlas si proceden de la entrada del usuario.
 - **Excepciones comprobadas (Checked Exceptions)**: Errores derivados de factores externos al código (fallos de ficheros `IOException`, bases de datos `SQLException`, redes). El compilador `javac` **obliga taxativamente** a capturarlas con `try-catch` o a declararlas en la cabecera del método con `throws`.

2.2 Captura Estructurada con Bloques `try-catch`

La estructura `try-catch` permite “probar” la ejecución de un bloque de código potencialmente peligroso. Si durante su ejecución se produce un error, la JVM detiene inmediatamente el bloque `try` y salta al bloque `catch`.

correspondiente para ejecutar la rutina de rescate:

```
import java.util.Scanner;
import java.util.InputMismatchException;

public class DivisionSegura {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Introduce el numerador (entero): ");
            int num = sc.nextInt();

            System.out.print("Introduce el denominador (entero): ");
            int den = sc.nextInt();

            int cociente = num / den; // Peligro: ArithmeticException si den == 0
            System.out.printf("Resultado exacto: %d / %d = %d%n", num, den, cociente);

        } catch (ArithmeticException e) {
            System.out.println("❌ Error matemático: ¡No es posible dividir un número entero por cero!");
        } catch (InputMismatchException e) {
            System.out.println("❌ Error de formato: Debes introducir dígitos enteros válidos.");
        }

        System.out.println("→ El programa continúa su ejecución normal sin estrellarse.");
        sc.close();
    }
}
```

Captura Múltiple Moderna (*Multi-catch* - Java 7+)

Si varios tipos de excepción deben recibir idéntico tratamiento, podemos agruparlas en un único bloque `catch` separadas por la barra vertical (`|`):

```
try {
    procesarDatos();
} catch (ArithmeticException | NumberFormatException e) {
    System.out.println("Error numérico detectado: " + e.getMessage());
}
```

El Orden de los bloques `catch` Importa

Las excepciones se evalúan de arriba abajo. Siempre debes colocar los `catch` de clases hijas más específicas primero (por ejemplo, `InputMismatchException`) y las clases padre más genéricas después (por ejemplo, `Exception`). Si colocas `catch (Exception e)` al principio, capturará cualquier error y los bloques inferiores se convertirán en código inalcanzable (*unreachable code*), provocando un error de compilación.

2.3 El Bloque `finally` y su Garantía de Ejecución

El bloque `finally` se sitúa tras los bloques `catch`. Su característica definitoria es tajante: **su código se ejecutará SIEMPRE**, tanto si el bloque `try` finalizó con éxito, como si se produjo una excepción que fue capturada, e incluso

si se produjo una excepción no capturada o se ejecutó una sentencia `return`:

```
Scanner sc = new Scanner(System.in);
try {
    System.out.print("Introduce un número: ");
    int n = sc.nextInt();
    System.out.println("Número al cuadrado: " + (n * n));
} catch (Exception e) {
    System.out.println("Ha ocurrido una incidencia: " + e.getMessage());
} finally {
    // Se ejecuta INCONDICIONALMENTE
    System.out.println("Limpiando y liberando recursos del sistema...");
}
```

2.4 La Sentencia Moderna `try-with-resources` (Java 7+)

Cerrar flujos de datos (`Scanner`, archivos, sockets de red o conexiones a bases de datos) manualmente en bloques `finally` suele requerir comprobaciones farragosas contra valores `null`.

Para resolverlo, Java introdujo la construcción `try-with-resources`. Cualquier objeto que implemente la interfaz `java.lang.AutoCloseable` puede declararse dentro de los paréntesis del `try (...)`, y la JVM garantizará su **cierre automático e inmediato** en cuanto el flujo abandone el bloque:

```
import java.util.Scanner;

public class TryWithResourcesDemo {
    public static void main(String[] args) {
        // El Scanner se cerrará automáticamente al salir del bloque try
        try (Scanner teclado = new Scanner(System.in)) {
            System.out.print("Escribe tu nombre: ");
            String nombre = teclado.nextLine();
            System.out.println("Hola, " + nombre);
        } catch (Exception e) {
            System.out.println("Incidencia durante la lectura: " + e.getMessage());
        }
        // Aquí 'teclado' ya ha sido cerrado de forma segura y liberado
    }
}
```

2.5 Lanzamiento y Propagación: `throw` y `throws`

A menudo, un método detecta que los datos que ha recibido violan las precondiciones necesarias para operar (por ejemplo, una edad negativa o una cuenta bancaria sin fondos suficientes). En lugar de retornar un valor mágico erróneo como `-1`, el método debe **lanzar una excepción** formal:

- **`throw` (en imperativo):** Dispara una instancia concreta de excepción dentro del cuerpo de un método.
- **`throws` (en indicativo):** Declara en la firma de un método las excepciones comprobadas que dicho método puede llegar a disparar hacia quien lo invoque.

```

public class BancoSeguro {
    // Declaramos que el método puede arrojar una excepción ante datos inválidos
    public static void retirarDinero(double saldoActual, double cantidad) throws
    IllegalArgumentException {
        if (cantidad ≤ 0) {
            // Instanciamos y arrojamos la excepción
            throw new IllegalArgumentException("La cantidad a retirar debe ser
            estrictamente positiva.");
        }
        if (cantidad > saldoActual) {
            throw new IllegalArgumentException("Saldo insuficiente. Saldo disponible: "
            + saldoActual);
        }

        System.out.println("Retirada completada. Nuevo saldo: " + (saldoActual -
        cantidad));
    }

    public static void main(String[] args) {
        try {
            retirarDinero(100.0, 150.0);
        } catch (IllegalArgumentException e) {
            System.out.println("Transacción rechazada: " + e.getMessage());
        }
    }
}

```

El Anti-Patrón “Catch and Swallow” (Atrapar y Silenciar)

Uno de los hábitos más perniciosos de los desarrolladores noveles es dejar bloques `catch` completamente vacíos:

```

try {
    operacionCritica();
} catch (Exception e) {
    // Bloque vacío: ¡EL MAYOR ERROR DE LA INGENIERÍA DE SOFTWARE!
}

```

Hacer esto oculta los fallos bajo la alfombra: el programa fallará silenciosamente, corromperá datos en memoria y cuando la aplicación colapse minutos después, resultará imposible rastrear el origen del fallo. **Como mínimo, imprime siempre un mensaje o invoca `e.printStackTrace()`.**

3. Sentencias de Repetición: Bucles en Java

Un **bucle** es una estructura de control que repite la ejecución de un bloque de código mientras una determinada condición booleana se evalúe a `true`. Cada repetición del bloque recibe el nombre de **iteración**.

Java proporciona tres tipos fundamentales de bucles tradicionales más una variante optimizada para colecciones:

Bucles en Java

- while → Comprobación al inicio (de 0 a N iteraciones)
- do-while → Comprobación al final (de 1 a N iteraciones – ideal para menús)
- for → Control de iteraciones conocido con contador
- for-each → Recorrido simplificado de arrays y colecciones

3.1 El Bucle `while` (Comprobación Previa)

El bucle `while` evalúa la condición antes de cada iteración. Si la condición es `false` desde el principio, el cuerpo del bucle **no se ejecutará ni una sola vez**:

```
while (condicion) {
    // Instrucciones que se repiten mientras condicion sea true
    // DEBE existir algún mecanismo que modifique las variables de la condición
}
```

```
int contador = 1;
while (contador ≤ 5) {
    System.out.println("Iteración número: " + contador);
    contador++; // Modificación obligatoria del estado
}
```

El Patrón Bucle Centinela (*Sentinel Loop*)

Se utiliza cuando no sabemos cuántos datos introducirá el usuario y utilizamos un valor especial (el “centinela”, por ejemplo el `0` o `-1`) para indicar el final de la serie:

```
import java.util.Scanner;

public class BucleCentinelaDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int suma = 0;
        int conteo = 0;

        System.out.println("Introduce números enteros a sumar (teclea 0 para finalizar):");
        int numero = sc.nextInt(); // Lectura previa

        while (numero ≠ 0) { // '0' actúa como valor centinela
            suma += numero;
            conteo++;
            System.out.print("Siguiendo número (0 para salir): ");
            numero = sc.nextInt(); // Lectura dentro del bucle
        }

        System.out.printf("Has introducido %d números. Suma acumulada: %d\n", conteo, suma);
        sc.close();
    }
}
```

Peligro de Bucle Infinito

Si olvidas actualizar la variable de control dentro del cuerpo del bucle (o si la condición jamás puede llegar a ser `false`), el programa entrará en un **bucle infinito**: la CPU se pondrá al 100% de uso y la aplicación quedará completamente congelada. En IntelliJ IDEA puedes detener un programa en bucle infinito pulsando el botón rojo cuadrado (*Stop*) o el atajo `Ctrl + F2`.

3.2 El Bucle `do-while` (Comprobación Posterior)

En el bucle `do-while`, el bloque de código se ejecuta **primero** y la condición se evalúa **después**. Esto garantiza que las instrucciones se ejecutarán **siempre al menos una vez**:

```
do {
    // Bloque de instrucciones (se ejecuta al menos una vez)
} while (condicion); // ¡Ojo al punto y coma final obligatorio!
```

La Aplicación Estrella: Menús Interactivos y Validación de Entrada

El escenario predilecto de `do-while` es mostrar un menú al usuario y forzarle a elegir una opción válida:

```
import java.util.Scanner;

public class MenuInteractivaDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int opcion;

        do {
            System.out.println("\n=== PANEL DE CONTROL DAM ===");
            System.out.println("1. Ver perfil de usuario");
            System.out.println("2. Consultar notas del módulo");
            System.out.println("3. Descargar boletín de ejercicios");
            System.out.println("0. Salir de la aplicación");
            System.out.print("Elige una opción (0-3): ");

            opcion = sc.nextInt();

            if (opcion < 0 || opcion > 3) {
                System.out.println("⚠ Opción incorrecta. Por favor, selecciona un
número entre 0 y 3.");
            }

        } while (opcion != 0); // Repite mientras el usuario no decida salir

        System.out.println("Sesión finalizada. ¡Hasta pronto!");
        sc.close();
    }
}
```

3.3 El Bucle `for` Clásico (Contador Determinado)

Cuando se conoce de antemano el número de repeticiones (o se desea recorrer un rango numérico determinado), el bucle `for` condensa toda la lógica de control en una sola línea canónica:

```
for (inicialización; condición; actualización) {
    // Instrucciones a iterar
}
```

1. **Inicialización:** Se ejecuta una única vez al arrancar el bucle (ej. `int i = 0;`). La variable declarada tiene ámbito (*scope*) local exclusivo al bucle.
2. **Condición:** Se evalúa antes de cada ciclo (ej. `i < 10;`). Si es `true`, entra al cuerpo; si es `false`, el bucle termina.
3. **Actualización:** Se ejecuta inmediatamente al final de cada iteración (ej. `i++`, `i += 2`, `i--`).

```
// Recorrido ascendente estándar de 1 a 10
for (int i = 1; i ≤ 10; i++) {
    System.out.print(i + " ");
}
System.out.println();

// Recorrido descendente (cuenta atrás)
for (int i = 10; i ≥ 0; i--) {
    System.out.print(i + " ... ");
}
System.out.println("¡Despegue! 🚀");

// Incremento a saltos (números pares entre 0 y 20)
for (int i = 0; i ≤ 20; i += 2) {
    System.out.print(i + " ");
}
System.out.println();
```

3.4 Patrones Algorítmicos con Bucles: Contadores, Acumuladores y Banderas

Casi todos los algoritmos iterativos se construyen sobre la combinación de tres patrones fundamentales de variables:

1. **Contador:** Variable entera que se incrementa en una cantidad fija (habitualmente `+1`) cada vez que ocurre un evento de interés:

```
int contadorPares = 0;
if (n % 2 == 0) contadorPares++;
```

2. **Acumulador (o Totalizador):** Variable que suma o multiplica valores variables en cada ciclo para obtener un total agregado:

```
double totalVentas = 0.0;
totalVentas += precioArticulo;
```

3. **Bandera (Flag o Variable Booleana):** Variable de tipo `boolean` que comienza con un estado inicial (ej. `false`) y "se enciende" cuando se constata una condición especial:

```
boolean encontrado = false;
if (numero == 10) encontrado = true;
```

3.5 Sentencias de Ruptura de Flujo: `break` y `continue`

Java proporciona dos instrucciones especiales para alterar la secuencia normal de un bucle desde su interior:

- **`break`**: Aborta de forma fulminante la ejecución del bucle, saltando inmediatamente a la instrucción posterior al cierre de la llave `}`:

```
for (int i = 1; i ≤ 100; i++) {
    if (i == 42) {
        System.out.println("¡Elemento 42 localizado! Deteniendo búsqueda.");
        break; // Abandona el bucle inmediatamente
    }
}
```

- **`continue`**: Salta el resto de instrucciones de la iteración *actual* y avanza de inmediato al siguiente ciclo (en un `for`, ejecuta la cláusula de actualización `i++` y reevalúa la condición):

```
// Imprimir solo los números impares ignorando los pares
for (int i = 1; i ≤ 10; i++) {
    if (i % 2 == 0) {
        continue; // Pasa inmediatamente al siguiente número sin ejecutar el
    }
    System.out.println("Impar: " + i);
}
```

Bucles con Etiquetas (*Labeled Breaks*)

Cuando tenemos varios bucles anidados unos dentro de otros, un `break` simple solo rompe el bucle más interno en el que se encuentra. Si necesitamos abortar todos los bucles simultáneamente desde el nivel más profundo, utilizamos una **etiqueta (label)**:

```
buscarCoordenadas: // Etiqueta identificativa
for (int f = 0; f < 10; f++) {
    for (int c = 0; c < 10; c++) {
        if (matriz[f][c] == elementoBuscado) {
            System.out.printf("Encontrado en posición [%d, %d]\n", f, c);
            break buscarCoordenadas; // Rompe AMBOS bucles de golpe
        }
    }
}
```

3.6 Bucles Anidados (*Nested Loops*)

Un bucle anidado es sencillamente un bucle situado dentro del cuerpo de otro. El bucle interno completa todas sus iteraciones por cada ciclo individual del bucle externo.

Son esenciales para generar tablas de coordenadas, matrices bidimensionales y cuadrículas:

```
// Generación completa de las tablas de multiplicar del 1 al 9
for (int tabla = 1; tabla ≤ 9; tabla++) {
    System.out.println("\n--- TABLA DEL " + tabla + " ---");
    for (int mult = 1; mult ≤ 10; mult++) {
        System.out.printf("%d x %2d = %2d%n", tabla, mult, (tabla * mult));
    }
}
```

4. Arrays: Vectores y Matrices Multidimensionales

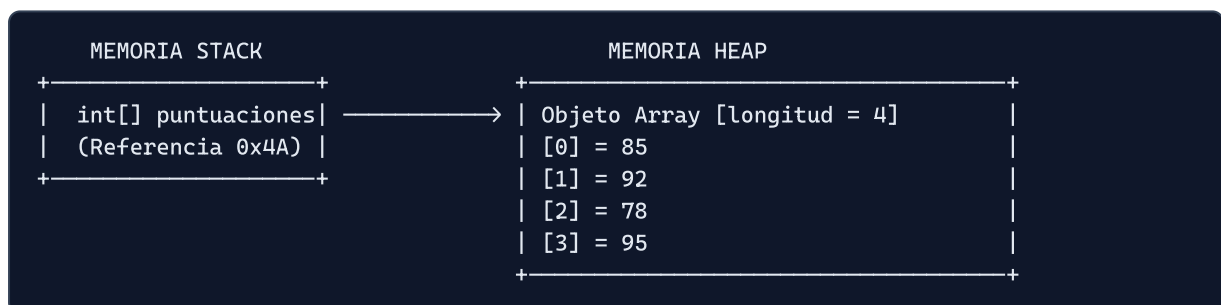
Hasta ahora, si un programa requiera gestionar las notas de 50 alumnos, nos veíamos obligados a declarar 50 variables individuales (`nota1` , `nota2` , ..., `nota50`). Esta aproximación es inescalable e inmanejable: imposibilita la iteración mediante bucles y llena el código de redundancia.

Para solventar esta limitación, Java implementa los **arrays** (también denominados *vectores*, *arreglos* o *tablas*).

4.1. Concepto y Representación en Memoria (Stack vs. Heap)

Un **array** es una estructura de datos homogénea (todos sus elementos son del mismo tipo) y estática (su tamaño se define en el momento de la instanciación y permanece inmutable durante todo su ciclo de vida).

En Java, a diferencia de lenguajes como C/C++, **los arrays son objetos de pleno derecho**. Esto implica una diferenciación crucial en la memoria de la JVM:



1. **Memoria Stack:** Aloja la variable de referencia (`puntuaciones`), cuyo contenido no son los datos numéricos, sino la dirección de memoria donde reside el objeto array en el Heap.
2. **Memoria Heap:** Aloja el bloque contiguo de memoria donde se almacenan físicamente los elementos indexados consecutivamente.

Tamaño Fijo e Inmutable

Una vez reservado un array de tamaño `N`, **no es posible ampliarlo ni reducirlo**. Si en tiempo de ejecución necesitas albergar `N + 1` elementos, deberás instanciar un nuevo array de mayor capacidad y volcar los elementos del antiguo en el nuevo (o utilizar colecciones dinámicas como `ArrayList`, tratadas en la Unidad 6).

4.2. Declaración, Instanciación e Inicialización

Existen diversas formas sintácticas de declarar y poblar un array en Java:

1. Declaración e Instanciación con Tamaño Fijo (Valores por Defecto)

Al reservar memoria con el operador `new`, Java inicializa automáticamente cada posición del array con el valor neutro o por defecto de su tipo:

- Tipos numéricos enteros (`byte`, `short`, `int`, `long`): `0`.
- Tipos numéricos reales (`float`, `double`): `0.0`.
- Booleanos (`boolean`): `false`.
- Caracteres (`char`): `'\u0000'` (carácter nulo).
- Tipos por referencia (`String`, objetos, wrappers): `null`.

```
// Declaración canónica (corchetes junto al tipo, estándar oficial de Java)
int[] temperaturas = new int[7];

// Asignación manual por índice (los índices comienzan SIEMPRE en 0)
temperaturas[0] = 18;
temperaturas[1] = 21;
temperaturas[6] = 19; // Última posición válida (7 - 1)
```

2. Inicialización Directa mediante Literales

Cuando conocemos los valores iniciales de antemano, podemos declarar e inicializar en una única sentencia compacta:

```
// Inicialización literal inline (forma preferida)
String[] diasSemana = {"Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo"};

// Inicialización anónima explícita (útil para pasar arrays como argumentos a métodos)
int[] factoresPrimos = new int[]{2, 3, 5, 7, 11};
```

4.3. Acceso a Elementos, Atributo `.length` y la Excepción `ArrayIndexOutOfBoundsException`

El acceso y modificación de cualquier elemento se realiza en tiempo constante `O(1)` indicando el índice entre corchetes `[índice]`:

- **Primer elemento:** `array[0]`.
- **Último elemento:** `array[array.length - 1]`.
- **Propiedad `.length`:** Es una variable constante de solo lectura pública que almacena el número total de celdas reservadas. (Nota: No lleva paréntesis; `length` es un atributo en arrays, mientras que en `String` es un método `length()`).

El Riesgo del Error Fuera de Límites

Si intentamos acceder a un índice negativo o a un índice mayor o igual a `.length`, la JVM aborta de inmediato la ejecución lanzando una excepción no comprobada de tipo `ArrayIndexOutOfBoundsException`:

```
int[] codigos = {101, 102, 103};

System.out.println(codigos[3]);
// ¡ERROR EN TIEMPO DE EJECUCIÓN!
// Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3 out of
// bounds for length 3
```

4.4. Recorridos: Bucle `for` Clásico vs. Bucle Mejorado (`for-each`)

Para procesar o mostrar los elementos de un vector, disponemos de dos variantes principales:

```
double[] calificaciones = {7.5, 4.2, 9.0, 6.8, 8.3};

// Variante A: Bucle for indexado clásico
// Obligatorio si necesitamos conocer o utilizar el índice numérico,
// o si deseamos modificar el contenido almacenado en el array.
for (int i = 0; i < calificaciones.length; i++) {
    System.out.printf("Alumno %d: %.1f%n", (i + 1), calificaciones[i]);
}

// Variante B: Bucle for-each (Enhanced For Loop)
// Recomendado cuando solo deseamos LEER secuencialmente todos los elementos.
// Sintaxis limpia: 'por cada elemento de tipo double dentro de calificaciones'
double sumaTotal = 0;
for (double nota : calificaciones) {
    sumaTotal += nota;
}
double media = sumaTotal / calificaciones.length;
System.out.printf("Media del grupo: %.2f%n", media);
```

El `for-each` es de Solo Lectura para Primitivos

En un bucle `for-each`, la variable iteradora (`nota` en el ejemplo anterior) es una **copia local** del valor en cada celda. Si ejecutas `nota = 10.0;` dentro del bucle, solo modificarás la variable temporal local; el contenido original del array `calificaciones` permanecerá intacto.

4.5. Algoritmos Esenciales sobre Arrays

El dominio de la programación estructurada exige comprender los patrones algorítmicos fundamentales para buscar, calcular y ordenar información en vectores.

1. Búsqueda de Mínimo, Máximo y Cálculo de Estadísticas

```
public class AnalisisVectores {
    public static void main(String[] args) {
        int[] datos = {45, 12, 89, 34, 12, 99, 23, 7};

        if (datos.length == 0) {
            System.out.println("El array está vacío.");
            return;
        }

        int maximo = datos[0];
        int minimo = datos[0];
        int acumuladorSuma = 0;

        for (int valor : datos) {
            if (valor > maximo) {
                maximo = valor;
            }
            if (valor < minimo) {
                minimo = valor;
            }
            acumuladorSuma += valor;
        }

        double media = (double) acumuladorSuma / datos.length;

        System.out.println("Valor Máximo: " + maximo);
        System.out.println("Valor Mínimo: " + minimo);
        System.out.printf("Promedio: %.2f%n", media);
    }
}
```

2. Inversión In-Place de un Array

Invertir los elementos sin crear un segundo array, intercambiando los extremos hacia el centro (técnica de dos punteros):

```
int[] valores = {1, 2, 3, 4, 5, 6};

int izquierda = 0;
int derecha = valores.length - 1;

while (izquierda < derecha) {
    // Intercambio (swap) utilizando variable auxiliar temporal
    int temporal = valores[izquierda];
    valores[izquierda] = valores[derecha];
    valores[derecha] = temporal;

    izquierda++;
    derecha--;
}
// Ahora valores contiene: {6, 5, 4, 3, 2, 1}
```

3. Algoritmo de Búsqueda Lineal vs. Búsqueda Binaria

- **Búsqueda Lineal:** Recorre elemento a elemento hasta encontrar el objetivo o agotar el array. Complejidad en el peor caso: $O(n)$. Funciona en arrays desordenados.

- **Búsqueda Binaria (Dicotómica):** Divide el espacio de búsqueda a la mitad en cada paso. Complejidad en el peor caso: $O(\log n)$. **Requisito obligatorio:** El array debe estar previamente ordenado de forma ascendente.

```
// Búsqueda binaria manual
public static int busquedaBinaria(int[] arrayOrdenado, int objetivo) {
    int inicio = 0;
    int fin = arrayOrdenado.length - 1;

    while (inicio ≤ fin) {
        // Evita el desbordamiento de enteros frente a (inicio + fin) / 2
        int medio = inicio + (fin - inicio) / 2;

        if (arrayOrdenado[medio] == objetivo) {
            return medio; // Encontrado, devuelve el índice
        } else if (arrayOrdenado[medio] < objetivo) {
            inicio = medio + 1; // Descartar mitad izquierda
        } else {
            fin = medio - 1;    // Descartar mitad derecha
        }
    }
    return -1; // No encontrado
}
```

4. Algoritmo de Ordenación por Burbuja (Bubble Sort)

Aunque en entornos profesionales se utilizan algoritmos optimizados como Quicksort o TimSort (`Arrays.sort()`), el algoritmo de la burbuja es fundamental en el aprendizaje académico para comprender la manipulación iterativa de índices y el intercambio (*swap*).

En cada pasada completa, el elemento más pesado (mayor) asciende hacia el final del array como una burbuja de aire en el agua:

```
public class OrdenacionBurbujaOptimizada {
    public static void main(String[] args) {
        int[] vector = {64, 34, 25, 12, 22, 11, 90};
        int n = vector.length;
        boolean huboIntercambio;

        for (int i = 0; i < n - 1; i++) {
            huboIntercambio = false;

            // En cada pasada i, los últimos i elementos ya están colocados
            for (int j = 0; j < n - 1 - i; j++) {
                if (vector[j] > vector[j + 1]) {
                    // Swap
                    int temporal = vector[j];
                    vector[j] = vector[j + 1];
                    vector[j + 1] = temporal;
                    huboIntercambio = true;
                }
            }

            // Si en una pasada completa no se hizo ningún intercambio,
            // el array ya está completamente ordenado; salimos antes.
            if (!huboIntercambio) {
                break;
            }
        }

        System.out.println(java.util.Arrays.toString(vector));
        // Salida: [11, 12, 22, 25, 34, 64, 90]
    }
}
```

4.6. La Navaja Suiza: `java.util.Arrays`

La biblioteca estándar de Java proporciona la clase de utilidad `java.util.Arrays`, equipada con métodos estáticos de alto rendimiento:

Método	Propósito	Complejidad / Comportamiento
<code>Arrays.toString(arr)</code>	Convierte el contenido del array a un String legible <code>"[elem1, elem2, ...]"</code>	Evita imprimir el identificador de referencia tipo <code>"[I@4f023edb"</code>
<code>Arrays.sort(arr)</code>	Ordena el array in-place en orden ascendente natural	Utiliza Dual-Pivot Quicksort para primitivos ($O(n \log n)$)
<code>Arrays.binarySearch(arr, key)</code>	Busca un valor mediante búsqueda dicotómica	Requiere que <code>arr</code> esté ordenado. Si existe devuelve <code>index ≥ 0</code> ; si no, <code>-(puntoDeInsercion) - 1</code>
<code>Arrays.equals(arr1, arr2)</code>	Compara si dos arrays tienen la misma longitud y los mismos elementos en idéntico orden	Compara elemento a elemento en valor (no por identidad de memoria)
<code>Arrays.fill(arr, valor)</code>	Asigna el mismo valor a todas las posiciones del array	Útil para inicializar tableros con caracteres vacíos o <code>false</code>
<code>Arrays.copyOf(arr, nuevaLongitud)</code>	Clona el array con una nueva dimensión	Si es mayor, rellena con ceros/nulos. Si es menor, trunca
<code>Arrays.copyOfRange(arr, desde, hasta)</code>	Extrae un sub-array en el rango semiabierto <code>[desde, hasta)</code>	Crea una nueva instancia de array independiente
<code>Arrays.mismatch(arr1, arr2)</code>	Devuelve el primer índice donde difieren dos arrays	Devuelve <code>-1</code> si ambos son idénticos. Introducido en Java 9+

```
import java.util.Arrays;

public class UtilidadesArrays {
    public static void main(String[] args) {
        int[] base = {5, 2, 8, 1, 9};

        // 1. Representación textual
        System.out.println("Original: " + Arrays.toString(base)); // [5, 2, 8, 1, 9]

        // 2. Ordenación automática
        Arrays.sort(base);
        System.out.println("Ordenado: " + Arrays.toString(base)); // [1, 2, 5, 8, 9]

        // 3. Búsqueda binaria
        int pos = Arrays.binarySearch(base, 8);
        System.out.println("El número 8 está en el índice: " + pos); // 3

        // 4. Clonado y redimensionamiento
        int[] ampliado = Arrays.copyOf(base, 8);
        System.out.println("Ampliado: " + Arrays.toString(ampliado)); // [1, 2, 5, 8,
9, 0, 0, 0]
    }
}
```

4.7. Arrays Multidimensionales (Matrices)

Una **matriz** o array multidimensional en Java no es un bloque plano continuo de memoria en dos dimensiones, sino literalmente un **array cuyos elementos son referencias a otros arrays**.

```
matriz (int[][][])
+-----+-----+-----+
| fila 0 | fila 1 | fila 2 |
+-----+-----+-----+
|         |         |         |
|         |         | +--> [ 7, 8, 9 ] (int[])
|         | +-----> [ 4, 5, 6 ] (int[])
+-----> [ 1, 2, 3 ] (int[])
```

Declaración e Inicialización de Matrices Bidimensionales

```
// Matriz de 3 filas x 4 columnas inicializada con ceros
int[][] tablero = new int[3][4];

tablero[0][0] = 5; // Primera fila, primera columna
tablero[2][3] = 9; // Última fila, última columna

// Inicialización directa mediante literales anidados
int[][] coordenadas = {
    {10, 20},
    {30, 40},
    {50, 60}
};
```

Dimensiones y Propiedad `.length`

- `tablero.length`: Devuelve el número de **filas** (arrays de primer nivel).
- `tablero[i].length`: Devuelve el número de **columnas** de la fila `i` (longitud del array interno).

Recorrido Completo con Bucles Anidados

```
int[][] matriz = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

// Recorrido indexado
for (int i = 0; i < matriz.length; i++) {
    for (int j = 0; j < matriz[i].length; j++) {
        System.out.printf("%3d", matriz[i][j]);
    }
    System.out.println(); // Salto de línea tras completar cada fila
}

// Recorrido limpio con for-each anidado
for (int[] fila : matriz) {
    for (int celda : fila) {
        System.out.print(celda + " ");
    }
    System.out.println();
}
```

Matrices Asimétricas o Dentadas (*Jagged Arrays*)

Dado que cada fila es un array independiente, no existe ninguna restricción de que todas las filas compartan la misma longitud. Una matriz con filas de distintas capacidades se denomina **array dentado**:

```
// Declarar una matriz de 3 filas sin especificar las columnas todavía
int[][] piramide = new int[3][];

// Instanciar cada fila con capacidades distintas
piramide[0] = new int[1]; // Fila 0 tiene 1 elemento
piramide[1] = new int[3]; // Fila 1 tiene 3 elementos
piramide[2] = new int[2]; // Fila 2 tiene 2 elementos

// Inicialización literal directa de matriz dentada:
String[][] menuSemanal = {
    {"Café", "Tostada"},
    {"Zumo", "Huevos", "Pancake", "Fruta"},
    {"Té"}
};
```

Métodos de Apoyo para Matrices: `Arrays.deepToString()` y `Arrays.deepEquals()`

Si aplicamos `Arrays.toString(matriz)` a una matriz bidimensional, imprimirá las referencias de memoria de cada fila (p. ej. `[[I@2a139a55, [I@15db9742]`). Para inspeccionar o comparar matrices multidimensionales completas, debemos invocar las variantes profundas (*deep*):

```
System.out.println(Arrays.deepToString(matriz));
// Imprime correctamente: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

5. Procesamiento Eficiente de Cadenas: Bucles y `StringBuilder`

El procesamiento textual iterativo es uno de los focos más habituales de cuellos de botella de rendimiento y consumo innecesario de memoria en aplicaciones Java.

5.1. El Peligro de Concatenar con `+` Dentro de Bucles

Recordemos que la clase `java.lang.String` en Java es **inmutable**: una vez creado un objeto `String`, sus caracteres nunca pueden modificarse. Cualquier operación de concatenación genera un objeto completamente nuevo en la memoria.

Analicemos el siguiente patrón nocivo:

```
// ¡ANTIPATRÓN DE RENDIMIENTO CRÍTICO!
String resultado = "";
for (int i = 0; i < 100_000; i++) {
    resultado += i; // En cada iteración: nuevo objeto String en el Heap + copia de
    caracteres
}
```

¿Qué sucede en el Heap de la JVM?

1. En la iteración 1, se crea el String `"0"`.
2. En la iteración 2, se crea el String `"01"` copiando el contenido anterior.

3. En la iteración 10.000, se reserva memoria para 10.000 caracteres, se copian los 9.999 anteriores y se descarta el objeto previo, dejándolo como basura para el Garbage Collector.

Este bucle presenta una complejidad algorítmica cuadrática $O(n^2)$ en tiempo y memoria. Para 100.000 iteraciones, el programa puede tardar decenas de segundos y provocar pausas severas del recolector de basura.

5.2. `StringBuilder`: Mutabilidad y Eficiencia en Tiempo Lineal

Para manipular texto de forma dinámica y acumulativa dentro de estructuras repetitivas, Java proporciona la clase mutable `java.lang.StringBuilder`:

```
// SOLUCIÓN ÓPTIMA: O(n) lineal
StringBuilder sb = new StringBuilder(100_000); // Se recomienda preasignar capacidad estimada
for (int i = 0; i < 100_000; i++) {
    sb.append(i); // Modifica el buffer interno in-place sin generar nuevos objetos
}
String resultadoFinal = sb.toString(); // Se convierte a String únicamente al finalizar
```

Nota: Mientras que el bucle con `+` tardaría más de 15 segundos, la solución con `StringBuilder` concluye en menos de 10 milisegundos.

Métodos Fundamentales de `StringBuilder`

Método	Descripción	Ejemplo
<code>append(valor)</code>	Concatena cualquier tipo de dato primitivo u objeto al final del buffer	<code>sb.append("Total: ").append(42);</code>
<code>insert(offset, valor)</code>	Inserta texto en una posición específica desplazando el resto a la derecha	<code>sb.insert(0, "INICIO: ");</code>
<code>delete(inicio, fin)</code>	Elimina los caracteres en el rango semiabierto <code>[inicio, fin)</code>	<code>sb.delete(2, 5);</code>
<code>reverse()</code>	Invierte la secuencia completa de caracteres in-place	<code>sb.reverse();</code>
<code>setCharAt(index, ch)</code>	Modifica el carácter en un índice concreto	<code>sb.setCharAt(0, 'A');</code>
<code>length()</code>	Devuelve la cantidad de caracteres contenidos en el buffer	<code>sb.length();</code>
<code>toString()</code>	Genera el objeto inmutable <code>String</code> final a partir del buffer acumulado	<code>String s = sb.toString();</code>

¿`StringBuilder` o `StringBuffer`?

`StringBuffer` es idéntico en API a `StringBuilder`, pero todos sus métodos están sincronizados mediante cerrojos de concurrencia (`synchronized`), haciéndolo seguro en entornos multihilo (*thread-safe*), a costa de una penalización de rendimiento. En programación modular y algoritmos estándar monotarea, **usa siempre `StringBuilder`**.

5.3. Recorrido de Cadenas Carácter a Carácter

Para examinar o transformar una cadena carácter a carácter disponemos de dos estrategias principales:

```
String texto = "Programacion DAM 2026";

// Estrategia 1: Indexación mediante charAt()
for (int i = 0; i < texto.length(); i++) {
    char c = texto.charAt(i);
    if (Character.isDigit(c)) {
        System.out.println("Dígito detectado en posición " + i + ": " + c);
    }
}

// Estrategia 2: Conversión a vector de caracteres toCharArray() para bucle for-each
int vocales = 0;
for (char c : texto.toLowerCase().toCharArray()) {
    if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u') {
        vocales++;
    }
}
System.out.println("Total de vocales: " + vocales);
```

6. Programación Modular: Métodos y Funciones en Java

A medida que los problemas crecen en envergadura, escribir todo el flujo dentro del método `main` genera código espagueti monolítico, imposible de mantener, depurar o testear.

La **programación modular** se basa en el principio clásico de *divide y vencerás*: descomponer un sistema complejo en subprogramas más pequeños e independientes denominados **métodos** (o funciones/procedimientos).

6.1. Anatomía y Firma de un Método

Un método en Java se declara con la siguiente estructura sintáctica:

[modificadores] tipoRetorno nombreMetodo([parámetros]) [throws Excepciones]

```
// [modificadores] tipoRetorno nombreMetodo([parámetros]) [throws Excepciones]
public static double calcularPrecioFinal(double baseImponible, double tipoIva) {
    if (baseImponible < 0 || tipoIva < 0) {
        throw new IllegalArgumentException("Los importes no pueden ser negativos.");
    }
    double cuotaIva = baseImponible * (tipoIva / 100.0);
    return baseImponible + cuotaIva;
}
```

Desglose de componentes:

1. **Modificadores:** Determinan la visibilidad y el comportamiento. En esta unidad utilizaremos `public static` (métodos de clase accesibles globalmente que pueden invocarse directamente desde el `main` sin instanciar objetos con `new`).
2. **Tipo de Retorno:** Especifica el tipo de dato que el método devuelve a quien lo invocó (`int`, `double`, `String`, `int[]`, etc.). Si el método no devuelve ningún valor, su tipo de retorno es `void`.

3. **Nombre del Método:** Debe seguir la convención `lowerCamelCase` y comenzar con un verbo imperativo o descriptivo de la acción (`calcularTotal` , `esNumeroPrimo` , `imprimirReporte`).
4. **Parámetros Formales:** Lista separada por comas de variables receptoras con su tipo correspondiente.
5. **Sentencia `return` :** Transfiere el control de vuelta a la llamada, entregando el valor computado. Si el tipo de retorno es `void` , la palabra reservada `return;` es opcional y sirve para salir prematuramente del método.
6. **Firma del Método (*Method Signature*):** En Java, la firma está compuesta **exclusivamente por el nombre del método y la lista ordenada de los tipos de sus parámetros**. (El tipo de retorno **NO** forma parte de la firma).

6.2. El Gran Principio de Java: Paso de Parámetros Estrictamente POR VALOR

Uno de los errores conceptuales más extendidos entre programadores que inician en Java consiste en creer que los objetos o arrays se pasan «por referencia».

Regla de Oro Inquebrantable de la JVM

Java es estrictamente PASO POR VALOR (*Pass by Value*) en todos y cada uno de los casos, sin ninguna excepción.

La confusión radica en comprender qué es exactamente el «valor» que se copia en la llamada:

1. Paso de Tipos Primitivos (`byte` , `short` , `int` , `long` , `float` , `double` , `boolean` , `char`)

El valor copiado es el **dato binario numérico literal**. El método receptor recibe un duplicado en su propio marco de pila (*stack frame*). Cualquier modificación sobre el parámetro local no tiene ningún efecto sobre la variable original del invocador:

```
public static void intentarIncrementar(int numero) {
    numero = numero + 10; // Modifica la copia local de la pila
}

public static void main(String[] args) {
    int x = 5;
    intentarIncrementar(x);
    System.out.println(x); // Imprime 5 (x no se vio afectada)
}
```

2. Paso de Tipos por Referencia (Arrays y Objetos)

El valor copiado es **la dirección de memoria (el puntero de referencia) que apunta al objeto en el Heap**.

- Como ambas variables (la del método invocador y la del método receptor) poseen una copia de la misma dirección de memoria, **las modificaciones en el estado interno del objeto o en las celdas del array SÍ alteran el objeto original en el Heap**.
- Sin embargo, si dentro del método reasignas la variable parámetro haciéndola apuntar a una nueva dirección (`parametro = new int[ ... ]`), **esa reasignación NO afecta a la referencia original del invocador**.

```
public class DemostracionPasoPorValor {
    public static void modificarContenidoArray(int[] vector) {
        vector[0] = 999; // ¡Muta el contenido del array en el Heap!
    }

    public static void intentarReasignarArray(int[] vector) {
        vector = new int[]{100, 200, 300}; // Reasigna la copia local de la referencia
    }

    public static void main(String[] args) {
        int[] misNumeros = {1, 2, 3};

        modificarContenidoArray(misNumeros);
        System.out.println(misNumeros[0]); // Imprime 999 (el contenido cambió)

        intentarReasignarArray(misNumeros);
        System.out.println(misNumeros[0]); // Sigue imprimiendo 999 (la referencia NO
cambió)
    }
}
```

6.3. Sobrecarga de Métodos (*Overloading*)

La **sobrecarga** consiste en definir dos o más métodos en la misma clase con el **mismo nombre**, pero con **distinta firma** (diferente número de parámetros o diferentes tipos de parámetros).

Permite que una operación conceptualmente única se adapte a distintos conjuntos de datos sin inventar nombres forzados (`sumar2Enteros` , `sumar3Enteros` , `sumarDoubles`):

```
public class CalculadoraMatematica {

    // Sobrecarga 1: dos enteros
    public static int sumar(int a, int b) {
        return a + b;
    }

    // Sobrecarga 2: tres enteros
    public static int sumar(int a, int b, int c) {
        return a + b + c;
    }

    // Sobrecarga 3: dos números reales
    public static double sumar(double a, double b) {
        return a + b;
    }
}
```

El Tipo de Retorno No Distingue Sobrecargas

No es posible sobrecargar un método cambiando únicamente el tipo de retorno. Si intentas declarar `int procesar()` y `double procesar()` en la misma clase, el compilador emitirá un error indicando que el método ya está definido (`method already defined in class`).

6.4. Parámetros de Longitud Variable (*Varargs*)

Cuando un método necesita recibir una cantidad indeterminada de argumentos del mismo tipo (por ejemplo, calcular la media de 2, 5 o 20 números), Java dispone de los **varargs** (*variable arguments*), utilizando la sintaxis de puntos suspensivos `...`:

```
public class Estadisticas {
    // Internamente, Java empaqueta 'valores' en un array unidimensional int[]
    public static int calcularMaximo(int primerValor, int... restoValores) {
        int max = primerValor;
        for (int v : restoValores) {
            if (v > max) {
                max = v;
            }
        }
        return max;
    }

    public static void main(String[] args) {
        // Se puede invocar con cualquier número de argumentos:
        System.out.println(calcularMaximo(10));           // max: 10
        System.out.println(calcularMaximo(10, 45, 23, 89)); // max: 89
    }
}
```

Reglas sintácticas de Varargs:

1. Solo puede haber **un único** parámetro varargs en la lista de parámetros.
2. El parámetro varargs debe ser obligatoriamente el **último parámetro** de la declaración del método.

6.5. Ámbito (*Scope*) y Tiempo de Vida de las Variables

El ciclo de vida y la visibilidad de una variable están delimitados por el bloque de llaves `{ }` en el que ha sido declarada:

1. **Variables Locales:** Se crean en el Stack al entrar en el bloque donde residen y se destruyen instantáneamente al salir de él. No se inicializan automáticamente: intentar leer una variable local no inicializada produce un error de compilación.
2. **Parámetros del Método:** Variables locales inicializadas en el momento de la llamada con los valores suministrados por el invocador.
3. **Variables de Clase / Estáticas:** Declaradas a nivel de clase con `static`; permanecen en memoria durante toda la ejecución de la aplicación.
4. **Sombreado (*Shadowing*):** Si declaras una variable dentro de un bucle o bloque con el mismo nombre que una variable del método contenedor, Java emitirá un error de compilación por variable duplicada (a diferencia de lenguajes como C++ o JavaScript).

6.6. Recursividad: Concepto, Call Stack y Caso Base

La **recursividad** es una técnica de diseño algorítmico en la cual un método **se invoca a sí mismo** para resolver una instancia más reducida del mismo problema, hasta alcanzar un estado elemental no recursivo.

Todo método recursivo bien diseñado debe contener obligatoriamente dos partes:

1. **Caso Base (Condición de Parada):** Es el escenario trivial cuya solución se conoce de antemano sin realizar nuevas llamadas recursivas. Sin caso base, el método entraría en recursión infinita.
2. **Caso Recursivo:** Descompone el problema en un subproblema estrictamente más pequeño, garantizando el avance convergente hacia el caso base.

Anatomía de la Pila de Llamadas (*Call Stack*) con el Factorial

El factorial de un número entero positivo `n!` se define formalmente como:

- `0! = 1` y `1! = 1` (Casos Base).
- `n! = n × (n - 1)!` para `n > 1` (Caso Recursivo).

```
public class FactorialRecursivo {
    public static long factorial(int n) {
        if (n < 0) {
            throw new IllegalArgumentException("El factorial no está definido para negativos.");
        }
        // 1. CASO BASE
        if (n ≤ 1) {
            return 1;
        }
        // 2. CASO RECURSIVO
        return n * factorial(n - 1);
    }

    public static void main(String[] args) {
        System.out.println("Factorial de 4: " + factorial(4)); // 24
    }
}
```

Evolución del Call Stack para `factorial(4)`:

```
FASE DE DESCENSO (Apilado de marcos en el Call Stack):
[ factorial(1) ] → Retorna 1 (Alcanza el Caso Base)
[ factorial(2) ] → Espera: 2 * factorial(1)
[ factorial(3) ] → Espera: 3 * factorial(2)
[ factorial(4) ] → Espera: 4 * factorial(3)
[ main()       ] → Invocador inicial

FASE DE ASCENSO (Desapilado y resolución de multiplicaciones):
factorial(1) retorna 1
factorial(2) calcula 2 * 1 = 2 y retorna 2
factorial(3) calcula 3 * 2 = 6 y retorna 6
factorial(4) calcula 4 * 6 = 24 y retorna 24 a main()
```

El Desbordamiento de Pila (`StackOverflowError`)

La memoria Stack de la JVM es limitada (típicamente entre 1 MB y 2 MB por hilo). Si un método recursivo omite el caso base o la profundidad de las llamadas es excesiva, la JVM agota el espacio de la pila y lanza un error fatal irreparable: `java.lang.StackOverflowError`.

Recursividad vs. Iteración

Todo algoritmo recursivo puede transformarse en una versión iterativa equivalente utilizando bucles y estructuras de datos explícitas. La recursividad proporciona soluciones elegantes y directas para estructuras arborescentes, fractales y algoritmos de retroceso (*backtracking*), pero la iteración suele ser más eficiente en memoria y velocidad.

6.7. Robustez Modular: Precondiciones y Aserciones (`assert`)

Un método modular debe protegerse frente a entradas corruptas antes de ejecutar su lógica de negocio:

1. Validación de Argumentos Defensiva

Utiliza excepciones estándar de la biblioteca Java:

- `IllegalArgumentException`: Si un argumento contiene un valor inapropiado (números negativos, cadenas vacías).
- `NullPointerException` u `Objects.requireNonNull(obj, "El parámetro no puede ser nulo")`: Si se pasa una referencia nula.

```
import java.util.Objects;

public static double calcularRaizCuadrada(Double valor) {
    Objects.requireNonNull(valor, "El número a calcular no puede ser nulo.");
    if (valor < 0) {
        throw new IllegalArgumentException("No es posible calcular la raíz de un número negativo: " + valor);
    }
    return Math.sqrt(valor);
}
```

2. Aserciones en Java (`assert`)

Las aserciones se utilizan durante el desarrollo y depuración para comprobar invariantes internas del código que el programador asume como verdaderas en todo momento:

```
// Sintaxis: assert <condicionBooleana> : "Mensaje descriptivo";
assert saldoActual ≥ 0 : "Invariante rota: el saldo nunca puede ser negativo tras la operación";
```

Por defecto en producción las aserciones están **desactivadas** por razones de rendimiento. Para habilitarlas en tiempo de ejecución se debe añadir el parámetro `-ea` (*enable assertions*) en las opciones de la máquina virtual (en IntelliJ IDEA: *Run* → *Edit Configurations* → *VM Options* → escribir `-ea`).

7. Expresiones Regulares (Regex) en Java

Una **expresión regular** (*regex*) es una secuencia de caracteres que conforma un patrón de búsqueda formal. En el desarrollo de software profesional, las expresiones regulares son indispensables para dos tareas críticas:

1. **Validación de entradas:** Comprobar que un dato introducido por el usuario cumple un formato estricto (DNI, correo electrónico, código postal, contraseña segura).
2. **Extracción y limpieza de texto:** Aislar subcadenas específicas o formatear logs y ficheros de texto.

7.1. Metacaracteres y la Regla del Doble Escape en Java

En Java, las cadenas literales interpretan la barra invertida `\` como carácter de escape (`\n` es salto de línea, `\t` es tabulador). Por tanto, para enviar un metacarácter de expresión regular al motor de Regex, **debemos escapar la barra invertida dos veces (`\\`)**:

Metacarácter	Significado	En Java (<code>String</code>)
<code>.</code>	Cualquier carácter individual (excepto salto de línea).	<code>"."</code>
<code>\d</code>	Cualquier dígito numérico <code>[0-9]</code> .	<code>"\\d"</code>
<code>\D</code>	Cualquier carácter que NO sea dígito <code>[^0-9]</code> .	<code>"\\D"</code>
<code>\w</code>	Carácter alfanumérico (letras, dígitos y guion bajo <code>[a-zA-Z_0-9]</code>).	<code>"\\w"</code>
<code>\W</code>	Cualquier carácter que NO sea alfanumérico.	<code>"\\W"</code>
<code>\s</code>	Espacio en blanco (espacio, tabulador, retorno de carro).	<code>"\\s"</code>
<code>\S</code>	Cualquier carácter que NO sea espacio en blanco.	<code>"\\S"</code>
<code>^</code>	Coincidencia al inicio exacto de la cadena o línea.	<code>"^"</code>
<code>\$</code>	Coincidencia al final exacto de la cadena o línea.	<code>"\$"</code>

7.2. Cuantificadores, Conjuntos y Grupos

Los **cuantificadores** especifican cuántas veces consecutivas debe repetirse el elemento precedente:

Cuantificador	Significado	Ejemplo
<code>?</code>	Cero o una vez (opcional).	<code>"https?"</code> coincide con <code>http</code> y <code>https</code> .
<code>*</code>	Cero o más veces.	<code>"\\d+"</code> coincide con cadena vacía, <code>"5"</code> , <code>"123"</code> .
<code>+</code>	Una o más veces (al menos una aparición obligatoria).	<code>"\\d+"</code> requiere al menos un número.
<code>{n}</code>	Exactamente <code>n</code> veces consecutivas.	<code>"\\d{4}"</code> exactamente 4 dígitos (p. ej. un año).
<code>{n,}</code>	Al menos <code>n</code> veces.	<code>"\\w{8,}"</code> al menos 8 caracteres alfanuméricos.
<code>{n,m}</code>	Entre <code>n</code> y <code>m</code> veces (ambos inclusive).	<code>"\\d{1,3}"</code> entre 1 y 3 dígitos.

Conjuntos y Rangos:

- `[aeiou]`: Coincide con cualquiera de las vocales minúsculas.
- `[a-zA-Z]`: Coincide con cualquier letra del abecedario en mayúscula o minúscula.
- `[^0-9]`: Negación (cualquier carácter excepto dígitos).
- `(rojo|verde|azul)`: Alternancia (coincide con cualquiera de las opciones completas).

7.3. Operaciones Nativas en `String`: `matches()`, `replaceAll()` y `split()`

La clase `String` incorpora métodos de conveniencia para interactuar con expresiones regulares de forma inmediata:

```
// 1. matches(regex): Comprueba si TODA la cadena encaja exactamente con el patrón
boolean esCodigoPostal = "46001".matches("\\d{5}"); // true

// 2. replaceAll(regex, sustituto): Reemplaza todas las coincidencias del patrón
String textoLimpio = "Usuario: 1234, Clave: 5678".replaceAll("\\d+", "XXXX");
// textoLimpio → "Usuario: XXXX, Clave: XXXX"

// 3. split(regex): Trocea la cadena según el delimitador especificado por el patrón
String lineaCSV = "Juan;DAM,Valencia;25";
String[] campos = lineaCSV.split("[;,]"); // Divide por punto y coma o por coma
// campos → ["Juan", "DAM", "Valencia", "25"]
```

7.4. La API de Alto Rendimiento: `Pattern` y `Matcher`

Cuando se ejecutan comprobaciones de expresiones regulares en bucles masivos, invocar repetidamente `string.matches(regex)` es ineficiente, pues Java recompila internamente el patrón de texto en cada iteración.

Para optimizar el rendimiento y extraer datos complejos, utilizamos las clases de `java.util.regex`:

1. `Pattern`: Representa el patrón precompilado e inmutable en memoria.
2. `Matcher`: El motor de coincidencia que escanea un texto contra el `Pattern`.

```
import java.util.regex.Pattern;
import java.util.regex.Matcher;

public class ExtraccionPatrones {
    // Patrón precompilado como constante estática (se compila una sola vez en toda la
    // aplicación)
    private static final Pattern PATRON_PRECIO = Pattern.compile("(\\d+)€");

    public static void main(String[] args) {
        String catalogo = "Teclado: 25€, Ratón: 15€, Monitor: 180€";
        Matcher matcher = PATRON_PRECIO.matcher(catalogo);

        int sumaPrecios = 0;
        // matcher.find() avanza buscando la siguiente coincidencia en el texto
        while (matcher.find()) {
            // group(1) extrae el contenido del primer grupo entre paréntesis (\\d+)
            int precio = Integer.parseInt(matcher.group(1));
            sumaPrecios += precio;
            System.out.println("Artículo detectado con precio: " + precio + "€");
        }
        System.out.println("Coste total del catálogo: " + sumaPrecios + "€");
    }
}
```

7.5. Casos Prácticos Esenciales en el Entorno DAM

A continuación se detallan los patrones de validación más frecuentes en aplicaciones empresariales y académicas:

1. Validación de DNI Español (8 dígitos y letra de control)

```
public static boolean validarFormatoDNI(String dni) {
    if (dni == null) return false;
    // Expresión: 8 dígitos exactos seguidos de una letra (mayúscula o minúscula)
    String regexDNI = "^\\d{8}[A-Za-z]$";
    if (!dni.matches(regexDNI)) {
        return false;
    }

    // Comprobación algorítmica de la letra oficial
    String letrasValidas = "TRWAGMYFPDXBNJZSQVHLCKE";
    int numero = Integer.parseInt(dni.substring(0, 8));
    char letraEsperada = letrasValidas.charAt(numero % 23);
    char letraReal = Character.toUpperCase(dni.charAt(8));

    return letraReal == letraEsperada;
}
```

2. Validación de Correo Electrónico Profesional

```
public static boolean validarEmail(String email) {
    if (email == null) return false;
    // Caracteres alfanuméricos y puntos/guiones antes del @, dominio y extensión de 2
    a 6 letras
    String regexEmail = "^[\\w.-]+@[\\w.-]+\\. [a-zA-Z]{2,6}$";
    return email.matches(regexEmail);
}
```

3. Validación de Matrícula Española Moderna (4 números y 3 letras sin vocales)

```
public static boolean validarMatricula(String matricula) {
    if (matricula == null) return false;
    // En España las matrículas modernas no usan vocales ni las consonantes Ñ y Q para
    evitar confusiones
    String regexMatricula = "^\\d{4}[BCDFGHJKLMNPSTVWXYZ]{3}$";
    return matricula.toUpperCase().matches(regexMatricula);
}
```

8. Depuración Avanzada con IntelliJ IDEA

El desarrollo de software profesional no consiste en adivinar qué ocurre mediante impresiones constantes de `System.out.println()`. IntelliJ IDEA incorpora un depurador gráfico de última generación que permite congelar el tiempo de ejecución, inspeccionar la memoria y descubrir defectos lógicos al instante.

8.1. Breakpoints Condicionales: Detener Bucles en el Momento Exacto

Imagina un bucle que procesa 10.000 iteraciones y falla únicamente en la número 8.432. Si colocas un punto de interrupción normal, tendrías que pulsar «Continuar» más de 8.000 veces.

Cómo activar un Breakpoint Condicional en IntelliJ:

1. Haz clic con el botón izquierdo en el margen izquierdo (junto al número de línea) para colocar un punto rojo (*breakpoint*).

- Haz **click derecho** sobre el punto rojo.
- En la casilla **Condition**, escribe la expresión booleana en Java que debe cumplirse para que el programa se congele (por ejemplo, `i == 8432` o `calificaciones[i] < 0`).
- Pulsa **Done**. Inicia la ejecución con el botón de depuración (el icono del insecto verde o pulsando **Shift + F9**). El programa correrá a máxima velocidad y solo se detendrá cuando la condición sea verdadera.

```

15 |   for (int i = 0; i < datos.length; i++) {
• → 16 |       procesarDato(datos[i]); ← [Condición: i == 8432]
17 |   }

```

8.2. El Panel de Depuración: Pila de Llamadas, Variables y Expresiones en Caliente

Cuando el depurador detiene el programa, se abre la consola inferior dividida en áreas clave:

- Panel Frames (Call Stack):** Muestra la cadena de llamadas activa. En métodos recursivos, verás apilados todos los marcos de llamada pendientes (`factorial(1)`, `factorial(2)`, etc.). Al hacer clic sobre cualquier marco anterior, el editor salta al estado exacto del código en ese punto.
- Panel Variables:** Muestra el valor en vivo de cada variable local, parámetros y referencias a arrays en el Heap.
 - En arrays y matrices, al desplegar la flecha se visualiza cada celda indexada `[0], [1], [2] ...` con sus valores actuales.
- Evaluate Expression (Alt + F8):** Abre un diálogo interactivo donde puedes escribir cualquier expresión en Java, consultar el estado de objetos o invocar métodos sobre las variables en vivo sin alterar el código fuente.

8.3. Atajos de Teclado Universales para la Navegación en IntelliJ

Acción	Atajo de Teclado	Descripción Funcional
Debug Program	Shift + F9	Compila y lanza la aplicación conectada al motor de depuración.
Step Over	F8	Ejecuta la línea actual por completo y avanza a la siguiente sin entrar al interior de los métodos invocados.
Step Into	F7	Si la línea contiene una llamada a un método propio, se adentra en su primera línea para depurarlo paso a paso.
Step Out	Shift + F8	Ejecuta de golpe el resto del método actual y se detiene en la línea donde fue invocado.
Resume Program	F9	Reanuda la ejecución continua hasta toparse con el siguiente breakpoint.
Stop Debugger	Ctrl + F2	Aborta y finaliza de inmediato la sesión de ejecución.

9. ★ Be the Code: Desafíos de Traza Mental

Un programador competente es capaz de ejecutar mentalmente el código como si fuera la propia Máquina Virtual de Java. Analiza minuciosamente los siguientes tres retos sin utilizar el ordenador y determina la salida exacta.

Desafío 1: Bucles Anidados con Etiquetas y Saltos de Flujo

¿Cuál es la salida exacta producida por el siguiente código en la consola?

```
public class RetoTraza1 {
    public static void main(String[] args) {
        int contador = 0;

        bucleExterno:
        for (int i = 1; i ≤ 3; i++) {
            for (int j = 1; j ≤ 3; j++) {
                if (i == 2 && j == 2) {
                    continue bucleExterno;
                }
                if (i == 3 && j == 2) {
                    break bucleExterno;
                }
                contador += (i * j);
                System.out.printf("[%d,%d] ", i, j);
            }
        }
        System.out.println("→ Total: " + contador);
    }
}
```

► 🔍 Ver Resolución y Traza Paso a Paso

Desafío 2: Mutación de Arrays vs. Reasignación de Punteros

¿Qué valores imprimirá el programa en cada una de las líneas comentadas?

```
public class RetoTraza2 {
    public static void alterar(int[] datos, int valorSimple) {
        valorSimple = 500;
        datos[0] = 88;
        datos = new int[]{10, 20, 30};
        datos[0] = 99;
    }

    public static void main(String[] args) {
        int x = 100;
        int[] lista = {1, 2, 3};

        alterar(lista, x);

        System.out.println("x = " + x); // Línea 1
        System.out.println("lista[0] = " + lista[0]); // Línea 2
        System.out.println("longitud = " + lista.length); // Línea 3
    }
}
```

► 🔍 Ver Resolución y Traza de Memoria

Desafío 3: Traza del Call Stack en Recursividad No Lineal

Analiza el siguiente método recursivo y deduce qué imprime exactamente en la consola:

```
public class RetoTraza3 {
    public static void misterio(int n) {
        if (n ≤ 0) {
            return; // Caso Base
        }
        System.out.print(n + " ");
        misterio(n - 1);
        System.out.print(n + " ");
    }

    public static void main(String[] args) {
        misterio(3);
        System.out.println();
    }
}
```

► 🔍 Ver Resolución y Traza de la Pila

10. ¡No Hay Preguntas Tontas!

1. ¿Por qué en las expresiones regulares de Java tengo que escribir `\d` con doble barra en vez de `\d` como en otros lenguajes?

En Java, las cadenas entre comillas dobles `" ... "` procesan la barra invertida `\` como carácter de escape propio del compilador (por ejemplo `\n` para nueva línea o `\"` para comillas). Si escribes `"\d"`, el compilador de Java cree que intentas usar una secuencia de escape especial de cadena y dará error de compilación porque `\d` no existe en la especificación sintáctica del lenguaje Java. Para que el carácter `\` llegue literalmente al motor de expresiones regulares, debes escapararlo anteponiendo otra barra: `"\\d"`.

2. Si comparo dos arrays con `array1 == array2`, ¿por qué da `false` aunque tengan exactamente los mismos números?

Porque en Java el operador `==` sobre tipos de referencia compara **identidades de memoria (direcciones físicas en el Heap)**, no el contenido de las celdas. Si creas dos arrays independientes con `new int[]{1, 2}` y `new int[]{1, 2}`, residen en dos posiciones de memoria distintas; por tanto `array1 == array2` es `false`. Para comparar el contenido posición a posición debes usar `Arrays.equals(array1, array2)` (o `Arrays.deepEquals()` si son matrices multidimensionales).

3. Si Java pasa siempre todo por valor, ¿por qué cuando modifico una celda de un array dentro de un método se altera el array original fuera?

Porque el «valor» que se copia en la llamada cuando trabajas con arrays u objetos es **la dirección de memoria de la referencia**. Imagina que tienes una casa (el array en el Heap) y una llave con su dirección (la variable en el Stack). Al invocar al método, Java le entrega al método un duplicado exacto de esa llave. Ambas llaves abren la misma casa. Si el método entra y pinta las paredes (modifica `array[0]`), la casa queda pintada para todos. Pero si el método tira su duplicado y compra una casa nueva (`array = new int[ ... ]`), tu llave original sigue abriendo la casa de siempre.

4. ¿Por qué en los arrays la longitud es `.length` sin paréntesis, pero en las cadenas de texto es `.length()` con paréntesis?

En un array, `length` es un **campo o atributo público constante** propio de la estructura interna del objeto array en la JVM. En cambio, en la clase `java.lang.String`, `length()` es un **método público** que consulta internamente la cantidad de caracteres almacenados en su buffer interno de bytes.

5. ¿Qué diferencia práctica y conceptual existe entre las palabras reservadas `throw` y `throws`?

- **`throw` (conjunción imperativa en singular):** Es una sentencia de acción directa que **lanza** una instancia concreta de excepción en tiempo de ejecución (`throw new IllegalArgumentException("Dato inválido");`).
- **`throws` (declaración en plural):** Es una cláusula en la **cabecera o firma** de un método que advierte al compilador y a otros programadores de qué excepciones comprobadas (*checked exceptions*) es susceptible de provocar ese método (`public void cargarArchivo() throws IOException`).

6. Si `StringBuilder` es tan rápido, ¿por qué no lo usamos siempre en lugar del operador `+`?

Para operaciones simples como concatenar dos o tres variables en una única línea (`String saludo = "Hola " + nombre + "!";`), el compilador moderno de Java (desde Java 9+) optimiza automáticamente la expresión transformándola internamente en una llamada a `StringConcatFactory` de alto rendimiento. Usar `StringBuilder` es crítico y obligatorio **cuando concatenas texto dentro de bucles o de forma acumulativa e iterativa**, donde el compilador no puede predecir el número de vueltas.

7. ¿Por qué un método recursivo sin caso base termina con un error fatal `StackOverflowError` en lugar de congelarse en un bucle infinito?

Un bucle iterativo infinito (`while(true)`) reutiliza siempre el mismo marco de pila y simplemente consume ciclos de CPU al 100%. Sin embargo, cada invocación a un método (incluido el recursivo) reserva un nuevo bloque de memoria física en el **Call Stack** para almacenar sus variables locales y la dirección de retorno. Como la memoria Stack es finita (habitualmente 1 MB), tras unos miles de llamadas sucesivas la pila se desborda físicamente, provocando la terminación inmediata del programa por agotamiento de memoria.

8. ¿Por qué el bloque `finally` se ejecuta incluso si antes coloqué una sentencia `return` dentro del `try`?

La JVM está diseñada para garantizar que las tareas de limpieza crítica (cerrar conexiones a bases de datos, liberar descriptores de ficheros o desconectar sockets de red) se efectúen sin importar cómo finalice el bloque protegido. Si la JVM encuentra un `return` en el `try`, primero evalúa la expresión de retorno, suspende momentáneamente la salida del método, **ejecuta íntegramente el bloque `finally`**, y únicamente después concluye transfiriendo el control al invocador.

11. Resumen de la Unidad y Conexión con la Unidad 4

En esta Unidad 3 hemos consolidado las cuatro columnas maestras del paradigma estructurado y modular en Java:

PILAS DE LA PROGRAMACIÓN ESTRUCTURADA EN JAVA			
EXCEPCIONES	BUCLES	ARRAYS	MÉTODOS
Control robusto de fallos con try-catch-finally y recursos.	while, do-while, for, for-each, centinelas, flags, break y continue.	Vectores, tablas multidimensional, búsqueda y Arrays utility.	Modularidad, paso por valor puro, recursión.

El Salto Hacia la Programación Orientada a Objetos (Unidad 4)

Hasta este momento, nuestros métodos (`public static`) y nuestros datos (`int[]`, variables independientes) vivían separados: los métodos procesaban datos que les pasábamos como parámetros externos.

En la **Unidad 4: Clases y Objetos**, daremos el gran salto conceptual hacia la **Programación Orientada a Objetos (POO)**:

- Empaquetaremos los datos (atributos) y los métodos que operan sobre ellos en entidades únicas y coherentes llamadas **Clases**.
- Aprenderemos a crear nuestros propios tipos de datos complejos mediante constructores, encapsulación (`private` / `public`) y referencias a objetos en el Heap.
- Diremos adiós al modificador `static` en los métodos de negocio para abrazar el estado y comportamiento de las instancias reales.

12. Relación Curricular Oficial

Esta unidad cubre exhaustivamente los contenidos y criterios de evaluación prescritos en el currículo oficial del ciclo formativo **CFGS Desarrollo de Aplicaciones Multiplataforma (DAM)** para el módulo **0485: Programación**:

- **RA1. Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.**
 - CE a, b, c, e, f: Identificación de bloques de código estructurado, declaración de arrays homogéneos y multidimensionales, y gestión de excepciones estándar de la plataforma Java.
- **RA2. Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje.**
 - CE a, b, c, d, e, f: Implementación correcta de bucles `while`, `do-while` y `for`, optimización con banderas y centinelas, uso de sentencias de ruptura controlada y depuración avanzada paso a paso con IntelliJ IDEA.
- **RA3. Escribe y optimiza código estructurado aplicando los principios de la programación modular.**
 - CE a, b, c, d, e, f: Descomposición de problemas en métodos reutilizables, paso riguroso de parámetros por valor en Stack y Heap, sobrecarga de métodos, varargs y análisis de la pila en algoritmos recursivos.
- **RA6. Escribe programas que apliquen estructuras de almacenamiento de información en memoria.**
 - CE a, b, c, d, e, f: Creación y manipulación de vectores y matrices, algoritmos de ordenación y búsqueda, procesamiento de cadenas con `StringBuilder` y validación robusta con expresiones regulares.

13. Boletines de Ejercicios y Retos Prácticos

Para afianzar y llevar a la práctica los conceptos teóricos y técnicos desarrollados en esta unidad, dispones de una batería de 5 boletines graduados por dificultad:

Recurso	Nivel / Enfoque	Descripción
 Boletín 03: Nivel Inicial	Básico / Consolidación	12 ejercicios prácticos para asentar bucles, arrays elementales y métodos sencillos.
 Boletín 03: Inicial Resuelto	Soluciones Comentadas	Soluciones completas con código Java 25 LTS y explicaciones paso a paso.
 Boletín 03: Nivel Intermedio	Intermedio / Aplicado	12 ejercicios avanzados con matrices, algoritmos de ordenación, regex y recursividad.
 Boletín 03: Intermedio Resuelto	Soluciones Comentadas	Soluciones completas con trazas, capturas de excepciones y buenas prácticas.
 Boletín 03: Retos Algorítmicos (Extras)	Avanzado / Competitivo	6 problemas de alta exigencia algorítmica (CodeWars / Acepta el Reto) con pistas y análisis.